



1922
ECUADOR

ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA



INTRODUCCIÓN A LA CALIDAD DEL SOFTWARE

UNIDAD 1



Unidad



Introducción a la calidad

Temario



01

Conceptos de Calidad del Software



02

Evolución de la Calidad



03

Costos de la calidad



04

Factores y criterios de Calidad de Software

1.1 Conceptos de Calidad de Software



Que es Calidad ?

Capacidad de desarrollar, diseñar, manufacturar y mantener un producto de valor. Este producto debe ser el más económico, el más útil y satisfactorio para el consumidor final. (Kaoru Ishikawa, 1988)

La calidad del software muestra qué tan bueno y confiable es un producto. Para transmitir un ejemplo de título asociado, piense en un software funcionalmente correcto.

La calidad de producto de software implica que hay un modelo de calidad subyacente que define un conjunto de características de producto (García-Mireles et al., 2015; ISO, 2011).

"Capacidad de un producto de software para cumplir funciones esperadas bajo condiciones definidas." Pressman, R. S. (2014), Software Engineering: A Practitioner's Approach, 8va ed.

Conjunto de características que determinan la capacidad de un producto de software para satisfacer necesidades explícitas e implícitas, incluyendo funcionalidad, fiabilidad, usabilidad, eficiencia, mantenibilidad y portabilidad, evaluadas bajo condiciones específicas y alineadas con estándares como ISO 25010.



1.1 Conceptos de Calidad de Software



Que es Calidad ?

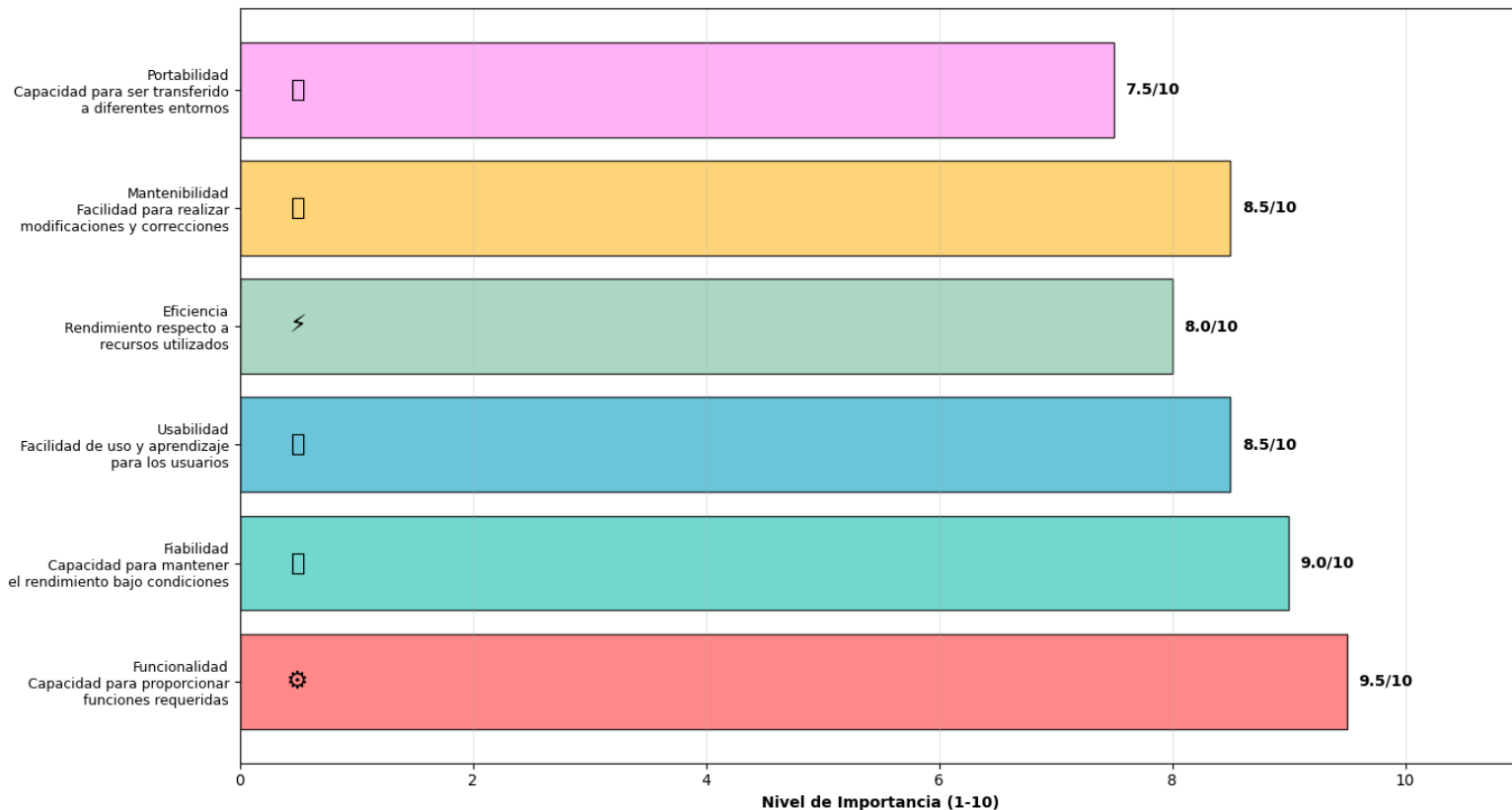
Según la norma ISO/IEC 25010, el concepto de calidad del software se divide en varias características.



1.1 Conceptos de Calidad de Software



▣ Atributos de Calidad del Software - Importancia Relativa



1.1 Conceptos de Calidad de Software



Origen de la CALIDAD

¿Qué es el Informe del Caos?

El informe del Caos viene siendo publicado por Standish group desde 1994 dando una visión sobre el fracaso o éxito de los proyectos.

En el informe del año 2015 han estudiado unos 50.000 proyectos de todo el mundo desde mantenimientos pequeños hasta gigantescos proyectos de reingeniería.

El Fracaso de un Proyecto

Para poder llegar a la causa raíz del fracaso en los proyectos, el PMI ha realizado en su serie de estudios Pulse of the Profession una consulta durante el mes de Mayo de este año 2014 sobre más de 2.000 profesionales de la gestión de proyectos.



Imagen de Tom Wigley

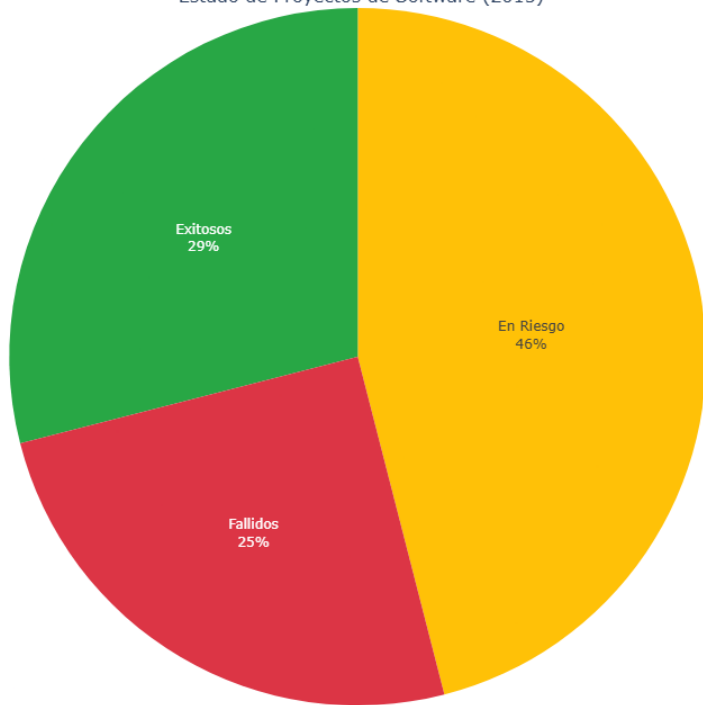
1.1 Conceptos de Calidad de Software



Informe Chaos Report 2015: Estado de los Proyectos de Software

Fuente: Standish Group | Análisis visual por IA | Fecha: 26/09/2025

Estado de Proyectos de Software (2015)



Tendencia de Éxito (1994-2015)



Factores Clave de Éxito

Liderazgo efectivo	9.2
Comunicación clara	8.8
Alcance bien definido	8.5
Compromiso del cliente	8.7
Gestión de cambios	8

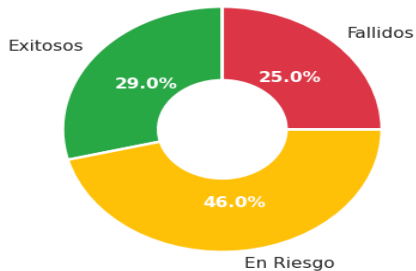
1.1 Conceptos de Calidad de Software



Informe Chaos Report 2015: Estado de los Proyectos de Software

Fuente: Standish Group | Análisis visual por IA | Fecha: 26/09/2025

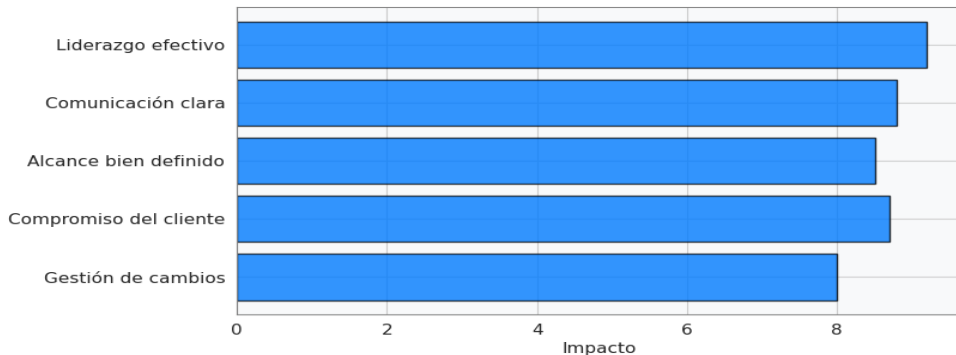
Estado de Proyectos (2015)



Tendencia Histórica de Proyectos (1994-2015)



Factores Clave de Éxito (Escala 0-10)



CONCLUSIÓN:

- Solo el 29% de los proyectos fueron exitosos en 2015.
- El 46% tuvieron problemas (retrasos, sobrecostos).
- La calidad del liderazgo y comunicación son factores críticos.

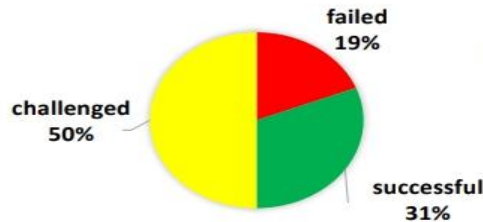
FUENTE: The CHAOS Report 2015 – Standish Group

1.1 Conceptos de Calidad de Software



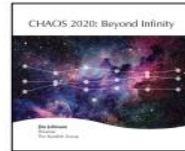
Project Success Quick Reference Card

Based on CHAOS 2020: Beyond Infinity Overview. January 2021, QRC by Henny Portman



Modern measurement (software projects)

Good Sponsor, Good Team, and Good Place are the only things we need to improve and build on to improve project performance.



The Good Place is where the sponsor and team work to create the product. It's made up of the people who support both sponsor and team. These people can be helpful or destructive. It's imperative that the organization work to improve their skills if a project is to succeed. This area is the hardest to mitigate, since each project is touched by so many people. Principles for a Good Place are:

- The Decision Latency Principle
- The Emotional Maturity Principle
- The Communication Principle
- The User Involvement Principle
- The Five Deadly Sins Principle
- The Negotiation Principle
- The Competency Principle
- The Optimization Principle
- The Rapid Execution Principle
- The Enterprise Architecture Principle



The Good Team is the project's workhorse. They do the heavy lifting. The sponsor breathes life into the project, but the team takes that breath and uses it to create a viable product that the organization can use and from which it derives value. Since we recommend small teams, this is the second easiest area to improve. Principles for a Good Team are:

- The Influential Principle
- The Mindfulness Principle
- The Five Deadly Sins Principle
- The Problem-Solver Principle
- The Communication Principle
- The Acceptance Principle
- The Respectfulness Principle
- The Confrontationist Principle
- The Civility Principle
- The Driven Principle



The Good Sponsor is the soul of the project. The sponsor breathes life into a project, and without the sponsor there is no project. Improving the skills of the project sponsor is the number-one factor of success – and also the easiest to improve upon, since each project has only one. Principles for a Good Sponsor are:

- The Decision Latency principle
- The Vision Principle
- The Work Smart Principle
- The Daydream Principle
- The Influence Principle
- The Passionate Principle
- The People Principle
- The Tension Principle
- The Torque Principle
- The Progress Principle



Successful project Resolution by Good Place Maturity Level:

highly mature	50%
mature	34%
moderately mature	23%
not mature	23%

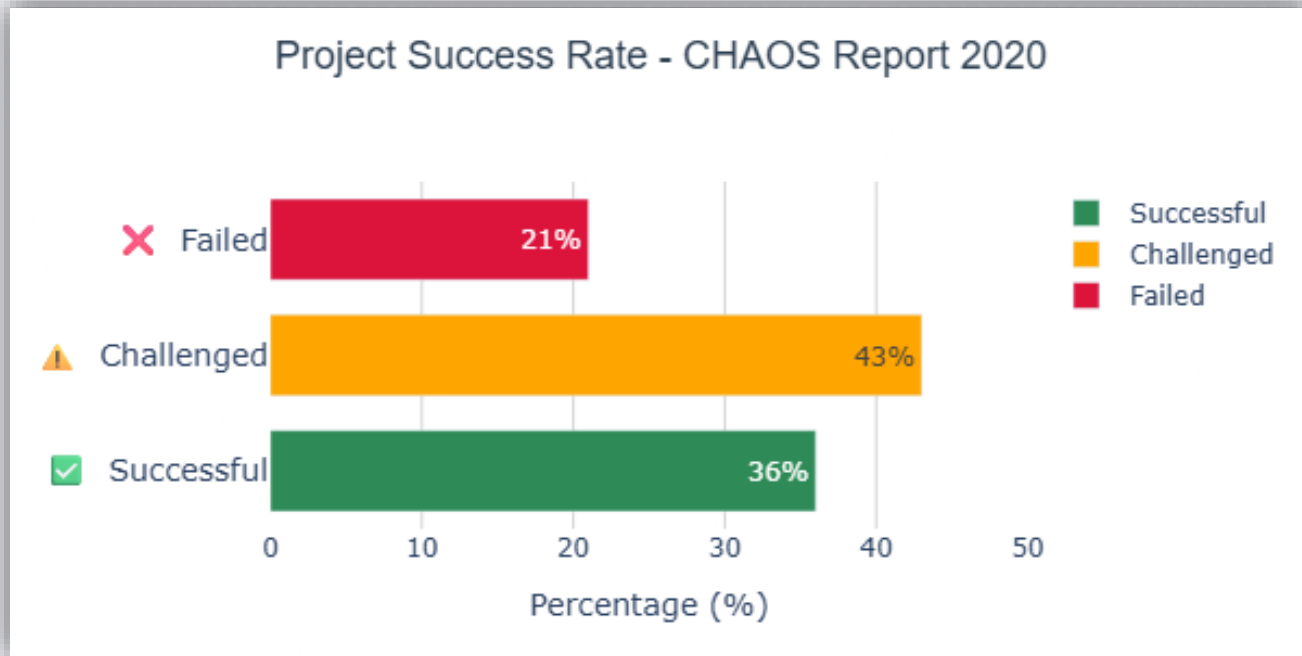
Successful project Resolution by Good Team Maturity Level:

highly mature	66%
mature	46%
moderately mature	21%
not mature	1%

Successful project Resolution by Good Sponsor Maturity Level:

highly mature	67%
mature	33%
moderately mature	21%
not mature	18%

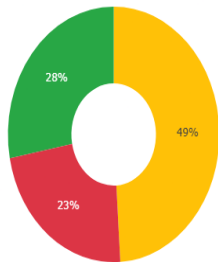
1.1 Conceptos de Calidad de Software



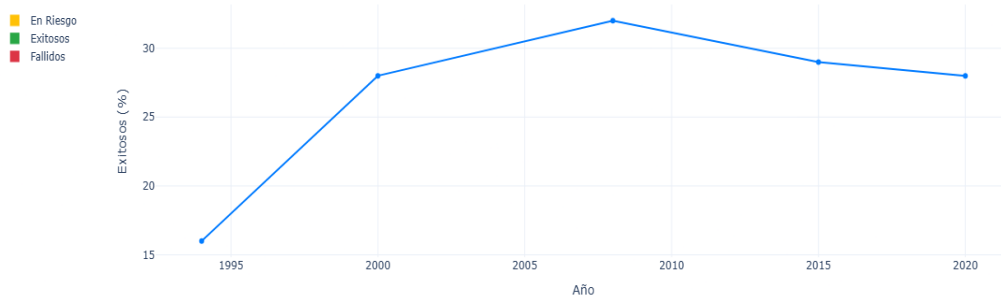
1.1 Conceptos de Calidad de Software



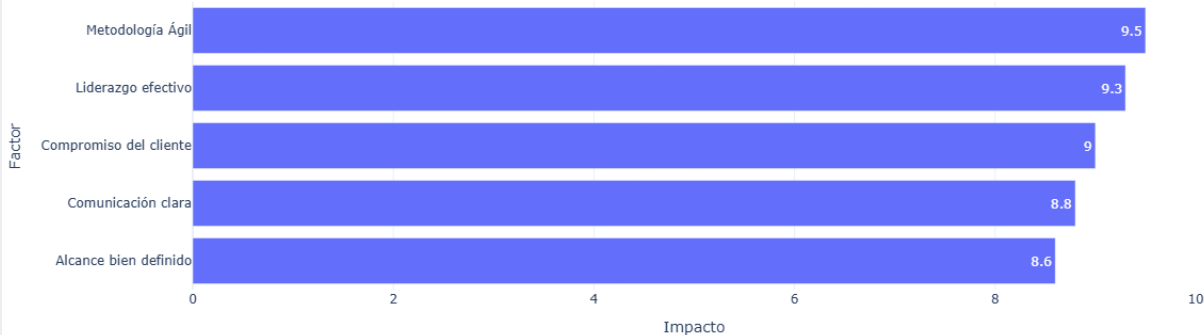
Estado de Proyectos (2020)



Tasa de Éxito de Proyectos (1994-2020)



Factores Clave de Éxito (2020)



1.1 Conceptos de Calidad de Software



"I do not worry whether something is cheap or expensive. I only worry if it is good. If it is good enough, the public will pay you back for it"

Walt Disney

1.1 Conceptos de Calidad de Software



- Los errores del software le salen muy caros a Estados Unidos: 59.500 millones de dólares al año.
- 50% de los fallos corresponde a los usuarios el resto a los programadores y vendedores.
- Las pruebas al inicio del ciclo reducirían los costos por fallos en 22.200 millones de dólares.

1.1 Conceptos de Calidad de Software



- 80% de los costos de desarrollo de programas se dedican a detectar y corregir defectos.
- La Academia de Ciencias de Estados Unidos pidió al Congreso una ley que atribuya a las casas de software una responsabilidad civil por daños y perjuicios a las empresa.

1.1 Conceptos de Calidad de Software



- El programa no esta probado
- El sistema operativo esta incompleto
- No están escritos los requisitos
- Estamos fuera de tiempo en un proyecto

1.1 Conceptos de Calidad de Software



La mayoría de los clientes busca **calidad al mejor precio**, sin embargo, lo que puede ser "excelente" para algunos, no lo es para otros. Cuando un individuo adquiere un producto o servicio, lo hace para satisfacer una necesidad, pero siempre espera que la "nueva adquisición" funcione como lo esperado, o al menos como se lo prometieron en el anuncio publicitario.

Muchas veces la calidad se paga, justificando de esta forma el dicho de que **"lo barato sale caro"**.

1.1 Conceptos de Calidad de Software



Cómo lograr que el consumidor se convierta en nuestro cliente

Calidad que se espera	Calidad que satisface	Calidad que deleita
Se da cuando existen propiedades y características que los consumidores dan por sentado que encontrarán en los productos o servicios. Cuando encuentran estas propiedades y características, los consumidores quedan satisfechos, pero cuando no las encuentran, quedan muy insatisfechos.	Se da cuando existen propiedades y características que los consumidores solicitan específicamente. Cuando están presentes estas propiedades y características, los consumidores quedan satisfechos, pero cuando no está presentes, quedan insatisfechos. La calidad que satisface cumple con las expectativas del consumidor, pero sin llegar a superarlas.	Se da cuando existen propiedades y características que los consumidores no solicitan porque no saben que puedan existir, pero que cuando están presentes y agradan, los consumidores quedan muy satisfechos; sin embargo, si no las encuentran, no quedan insatisfechos. La calidad que deleita supera las expectativas del consumidor.

1.1 Conceptos de Calidad de Software

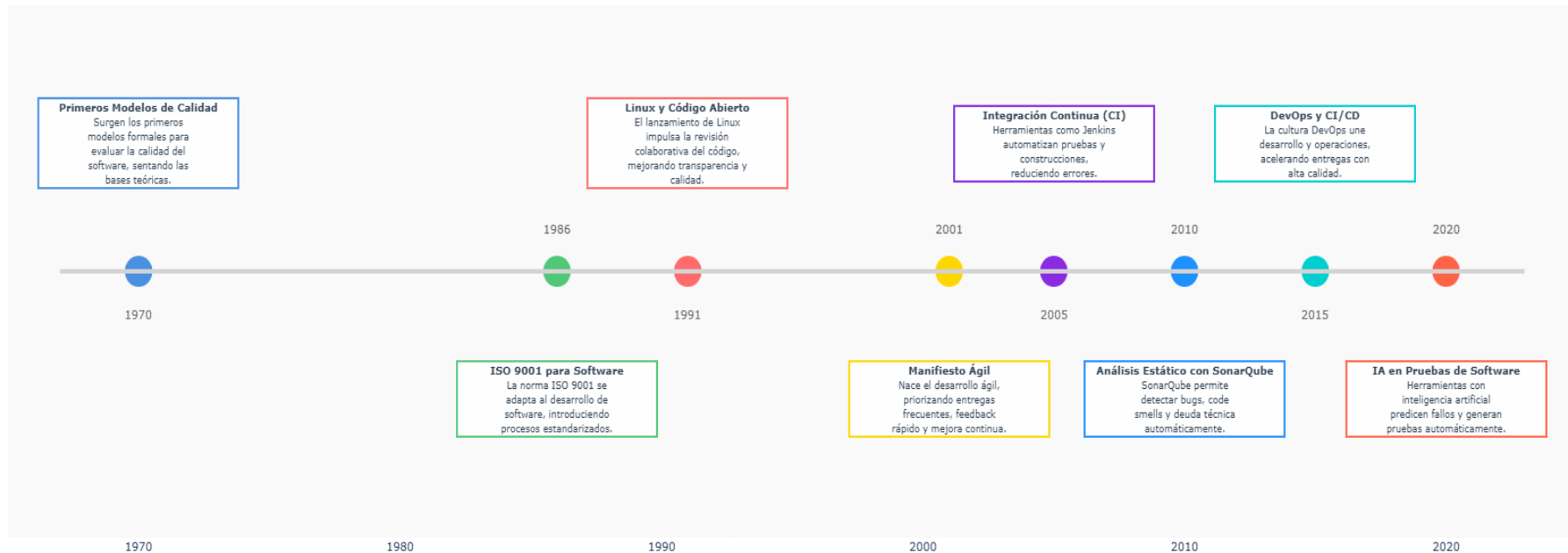


1.1 Conceptos de Calidad de Software



Evolución de la Calidad del Software

Hallazgos clave a lo largo de la historia del desarrollo de software



1.1 Conceptos de Calidad de Software



- ✓ En las primeras etapas, el software se desarrollaba sin metodologías formales.
- ✓ La calidad se medía principalmente por la ausencia de errores (corrección funcional).
- ✓ El enfoque era reaccionario: corregir errores después de su detección.
- ✓ Surgieron problemas como el "crisis del software" por proyectos que excedían presupuestos y plazos.

Ejemplo: Proyectos militares y espaciales con fallos costosos.

1.1 Conceptos de Calidad de Software



Ariane 5 Flight 501 (1996)



Qué pasó: El cohete europeo Ariane 5 explotó 37 segundos después del lanzamiento.

Causa: Un error de software por una conversión de dato errónea (de punto flotante a entero) en un sistema de referencia inercial.

Costo estimado: Alrededor de \$370 millones.

Lección: Se reutilizó código de Ariane 4 sin adaptarlo adecuadamente. Falta de pruebas bajo condiciones reales.

Therac-25 (1985–1987)



Qué pasó: Máquina de radioterapia que administró dosis mortales de radiación a pacientes debido a un error de software.

Causa: Errores de concurrencia (race condition) en el código; falta de validación y controles de hardware.

Consecuencias: Al menos 6 sobredosis, varias con resultado fatal.

Lección: Calidad del software = seguridad del paciente. Sin mecanismos de falla segura (fail-safe).

Patriot Missile Failure (Guerra del Golfo, 1991)



Qué pasó: El sistema antimisiles Patriot falló en interceptar un misil Scud iraquí en Dhahran, Arabia Saudita.

Causa: Un error de redondeo en el cálculo del tiempo (debido a aritmética de punto flotante con precisión limitada).

Consecuencias: Murieron 28 soldados estadounidenses.

Costo humano y político muy alto.

Lección: Pequeños errores numéricos pueden tener consecuencias catastróficas si no se validan los sistemas críticos.

Mars Climate Orbiter (1999)



Qué pasó: La nave espacial de la NASA se perdió al intentar orbitar Marte.

Causa: Confusión entre unidades: un equipo usó libras-fuerza y otro newtons en los cálculos de empuje.

Error de software/interfaz: El sistema no detectó ni corrigió la inconsistencia.

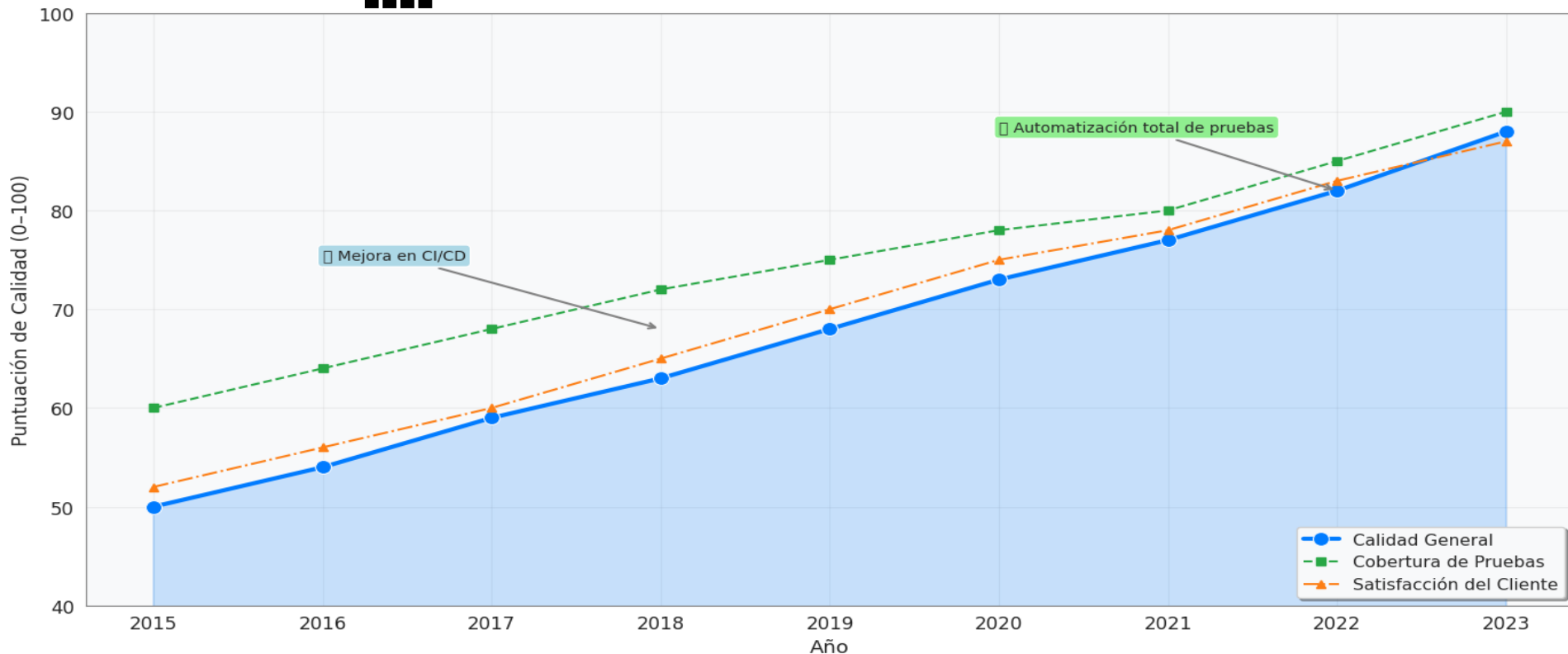
Costo: \$327 millones perdidos.

Lección: Necesidad de estandarización, verificación cruzada y gestión de datos precisa.

1.1 Conceptos de Calidad de Software



Evolución de la Calidad del Software (2015-2023)

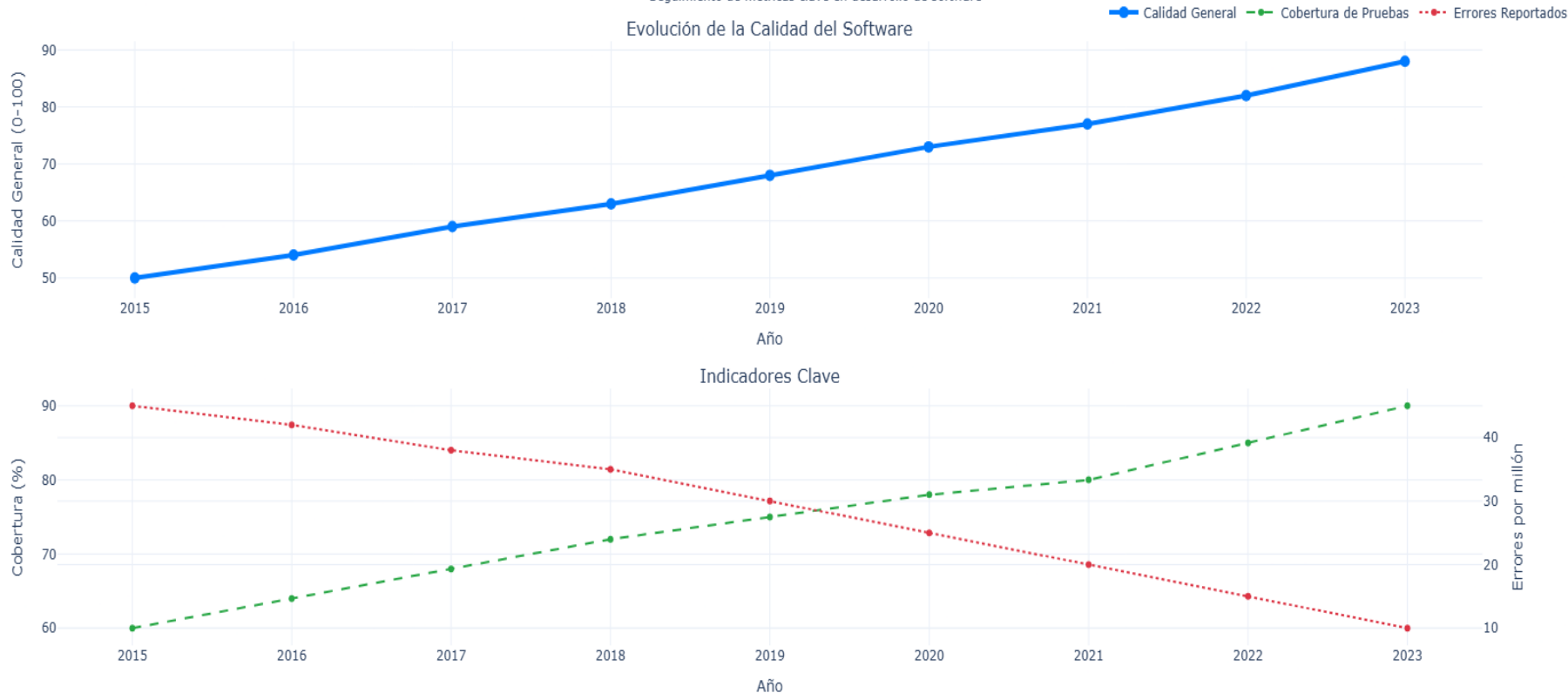


1.1 Conceptos de Calidad de Software

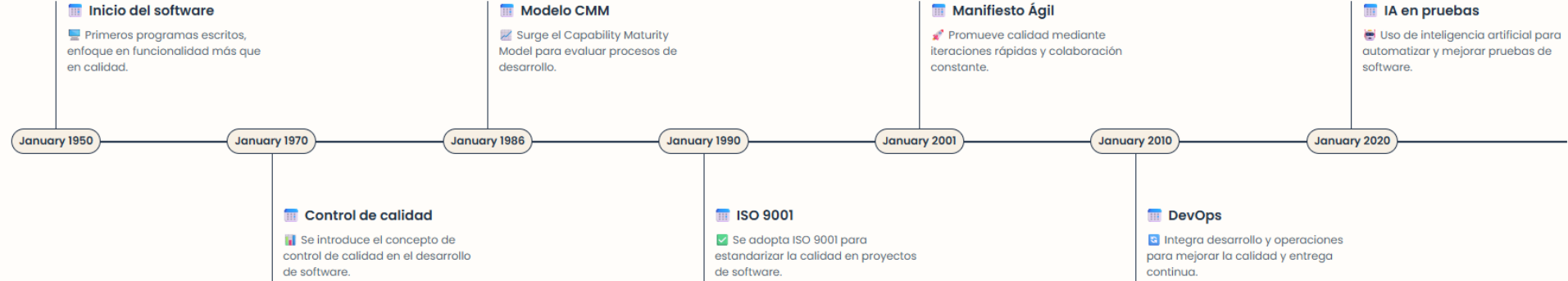


Evolución de la Calidad del Software (2015-2023)

Seguimiento de métricas clave en desarrollo de software



1.1 Conceptos de Calidad de Software



1.1 Conceptos de Calidad de Software



Hay que tomar en cuenta los siguientes puntos

- ☑ Los requisitos de Software son la base de las medidas de calidad
- ☑ Impacto estratégico. Oportunidad de ventaja competitiva.
- ☑ Planificación, fijación de objetivos, coordinación, formación y adaptación de toda la organización.
- ☑ “Una filosofía, una cultura, una estrategia, un estilo de gerencia de la empresa”.

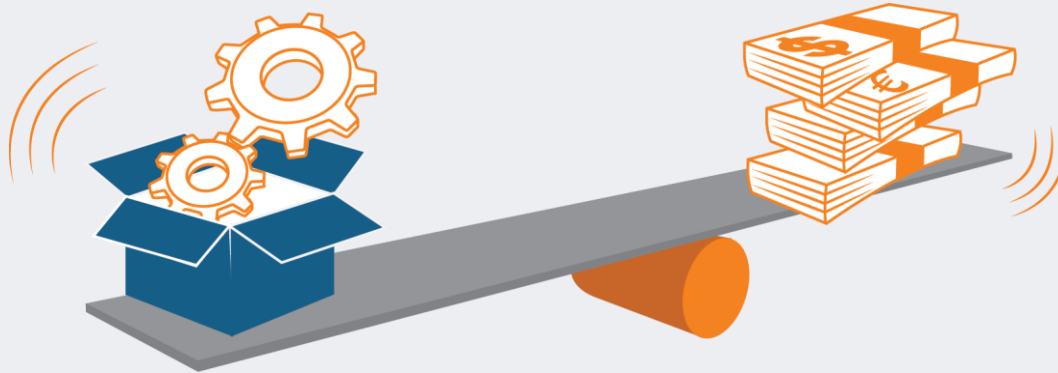
1.1 Conceptos de Calidad de Software



Definición del término "costo"

En forma similar al concepto de "**calidad**" el término "**costo**" también recibe distintas acepciones, y no siempre se emplea en forma correcta.

Podría tomarse como la definición que describe al término "**costo**" a la siguiente expresión: *"un costo es el esfuerzo económico que debe realizarse para alcanzar un objetivo"*.



1.1 Conceptos de Calidad de Software



Aplicando estos conceptos al sector del desarrollo software este análisis quedaría:



Costes de las pruebas: Costes derivados de probar el software para evitar que los defectos lleguen a producción.



Prevención: Costes derivados de mejorar los procesos de desarrollo (herramientas, técnicas, arquitectura, métodos, prácticas) y de capacitación de las personas (formación) para minimizar la inyección de defectos en el software.



Costes del fallo interno (retrabajo): Costes de búsqueda y corrección del defecto así como repetición de las pruebas derivados de haber encontrado un defecto antes de liberar el software. No sólo hay que tener en cuenta el coste de repetir la prueba que originó el fallo (retest) sino la posible regresión de pruebas y los costes asociados a un nuevo paso a producción que suelen ser los mayor cuantía.



Costes del fallo en producción: En este escenario no sólo hay que sumar los costes equivalentes al fallo interno, sino el daño producido al cliente, usuario o el propio negocio donde se utiliza el software. La pérdida de confianza y las posibles penalizaciones o sanciones suelen ser los mayores costes de este tipo de fallos.

1.1 Conceptos de Calidad de Software



Aplicando estos conceptos al sector del desarrollo software este análisis quedaría:



Costes de las pruebas: Costes derivados de probar el software para evitar que los defectos lleguen a producción.



Prevención: Costes derivados de mejorar los procesos de desarrollo (herramientas, técnicas, arquitectura, métodos, prácticas) y de capacitación de las personas (formación) para minimizar la inyección de defectos en el software.



Costes del fallo interno (retrabajo): Costes de búsqueda y corrección del defecto así como repetición de las pruebas derivados de haber encontrado un defecto antes de liberar el software. No sólo hay que tener en cuenta el coste de repetir la prueba que originó el fallo (retest) sino la posible regresión de pruebas y los costes asociados a un nuevo paso a producción que suelen ser los mayor cuantía.



Costes del fallo en producción: En este escenario no sólo hay que sumar los costes equivalentes al fallo interno, sino el daño producido al cliente, usuario o el propio negocio donde se utiliza el software. La pérdida de confianza y las posibles penalizaciones o sanciones suelen ser los mayores costes de este tipo de fallos.

1.1 Conceptos de Calidad de Software



Tipo de Costos de Software

- ☐ **Costos para asegurar la calidad o costos de conformidad**
 - Costos de Prevención
 - Costos de Evaluación
- ☐ **Costos por falta de calidad o costos de no conformidad**
 - Costos de fallas internas
 - Costos de fallas externas

1.1 Conceptos de Calidad de Software



Costos de prevención



Costos de todos aquellos esfuerzos para asegurar la Calidad del Software y prevenir defectos en todas las fases del Desarrollo de Software. Por ejemplo: **Aseguramiento de la Calidad**.

Costos de evaluación



Costo del esfuerzo para descubrir la condición de la Calidad del Software (evaluaciones planeadas). Por ejemplo: **Evaluación de proyectos** (revisión de especificaciones de requerimientos, diseño y componentes, verificaciones y validaciones en general, inspecciones, pruebas unitarias, de integración y de sistema), **auditorías de calidad del producto, evaluaciones externas, pruebas de productos adquiridos**.

Costos de fallas internas



Costo del esfuerzo para detectar y corregir problemas previos a que el usuario los detecte, por ejemplo: **corregir defectos** y el **retrabajo correctivo** en todas las etapas.

Costos de fallas externas

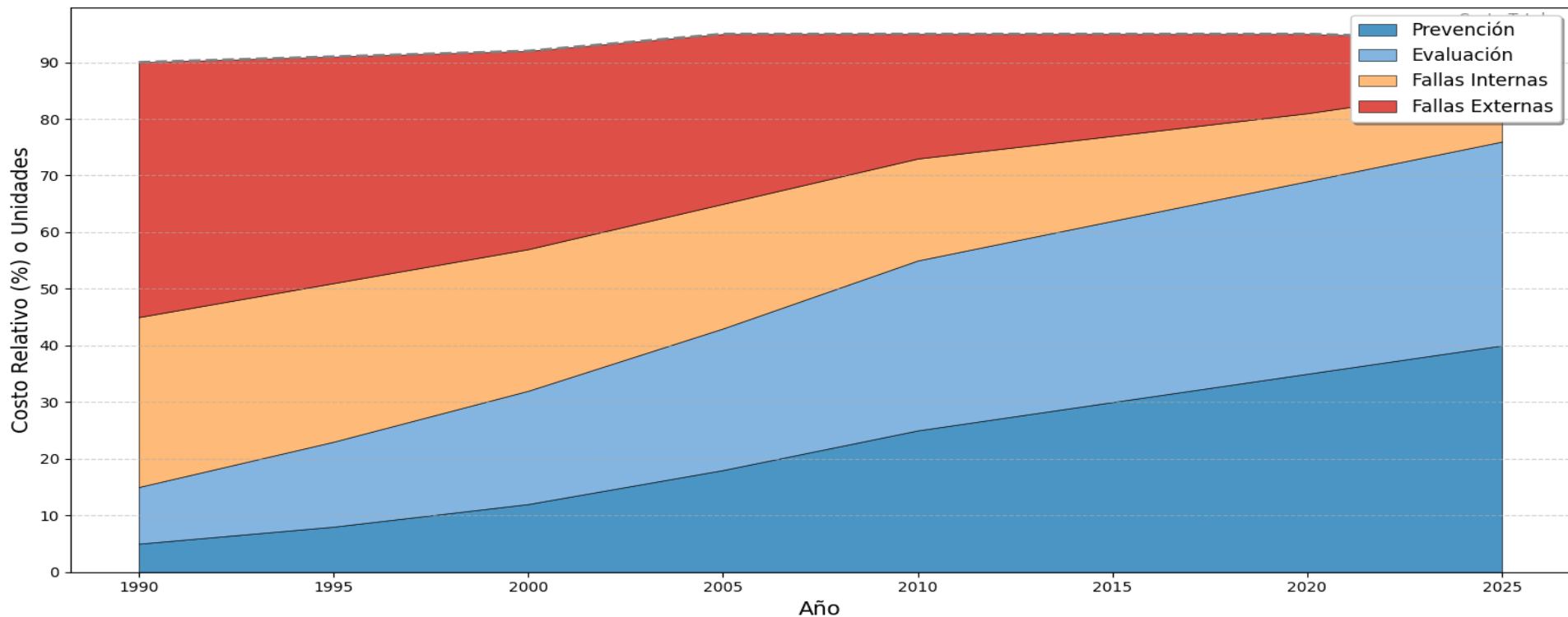


Costo del esfuerzo para corregir problemas que son detectados por el usuario. Esto incluye remoción de **fallas** (mantenimiento correctivo, retrabajo, codificación y prueba de defectos, compra de actualizaciones de productos), **soporte** (soporte técnico para responder por los defectos, preparación de apuntes de respuestas de soporte, investigación de las quejas del usuario), **compensación** (penalizaciones, reclamaciones, productos devueltos, reembolsos, descuentos, costos de garantía), **otros** (mala reputación, pérdida de ventas, usuario insatisfecho, pérdida de la confianza del cliente).

1.1 Conceptos de Calidad de Software



**Evolución de los Costos de Calidad en el Desarrollo de Software
(1990 - 2025)**

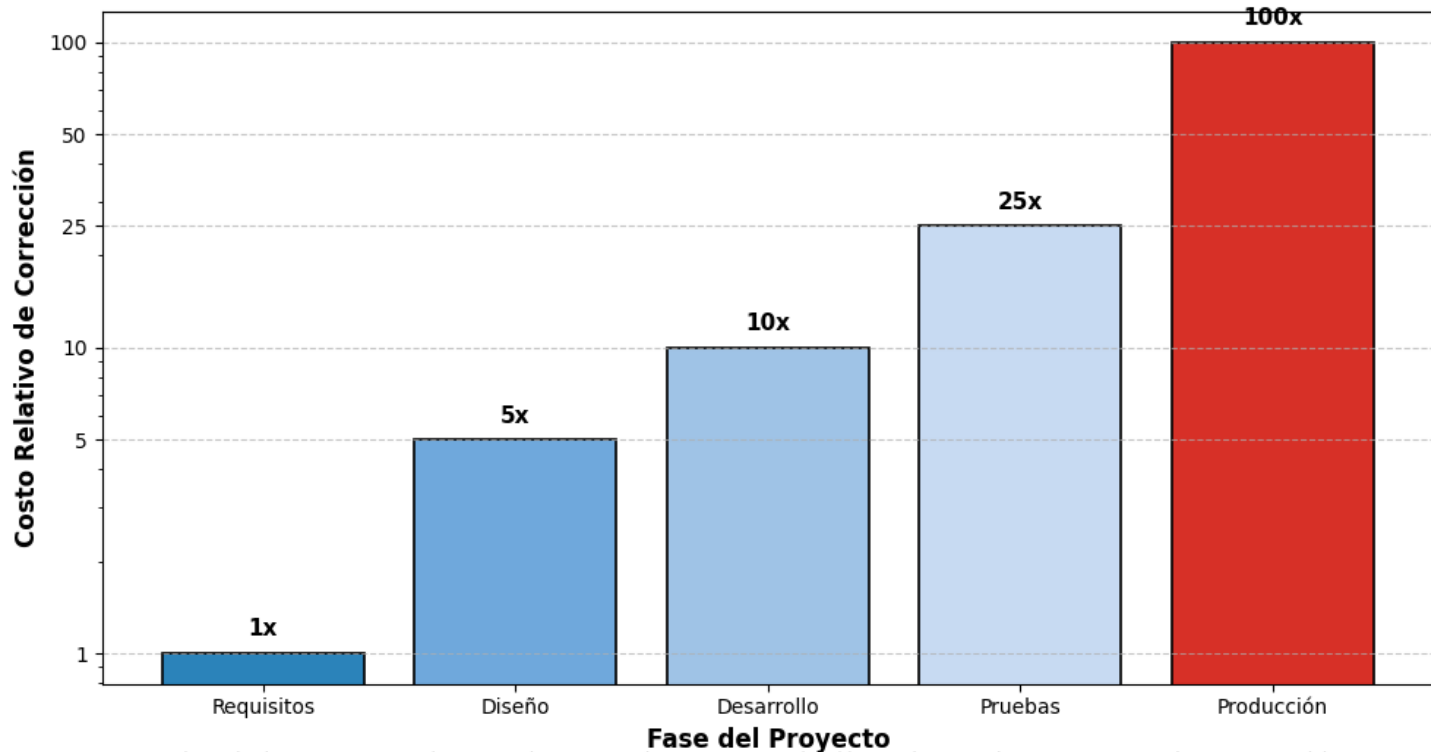


Fuente: Modelo de Costos de Calidad (CoQ) adaptado a software. Tendencia: mayor inversión en prevención reduce fallos externos.

1.1 Conceptos de Calidad de Software



**Costo Relativo de Corregir un Defecto
por Fase del Ciclo de Vida del Software**

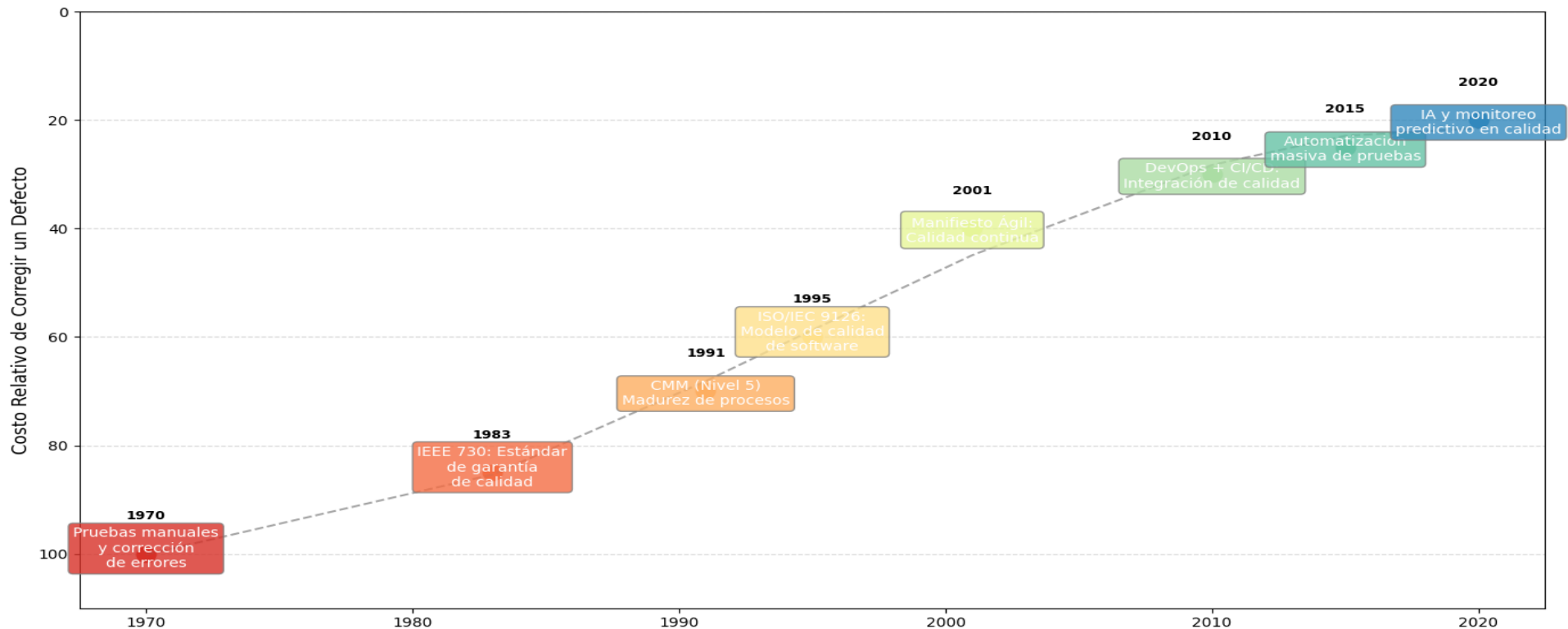


Fuente: Adaptado de IBM Systems Sciences Institute. Corregir errores en producción puede costar hasta 100 veces más que en requisitos.

1.1 Conceptos de Calidad de Software



Evolución Histórica de la Calidad del Software y su Impacto en los Costos (1970 - 2020)



Fuente: Adaptado de IEEE, ISO, CMMI, y estudios de IBM/NASA sobre costo de calidad.

1.1 Conceptos de Calidad de Software

Conclusión

La administración de la calidad total como una variable y estrategia de software, es la clave que siempre toman en cuenta en las empresas, las cuales necesitan verla y tratarla como corresponde.

Uno de los factores para promover un proceso de mejoramiento continuo de la calidad consiste en **recopilar, documentar** y usar la **información** sobre los costos relacionados con la calidad.

Es necesario que los costos de la calidad sean declarados formalmente por las empresas, parte de sus planes operacionales y sus sistemas presupuestarios, y que los ejecutivos los manejen como se debe.



1.1 Conceptos de Calidad de Software



Factores Clave de Calidad del Software

Funcionalidad

Cumple con lo que el usuario necesita: precisión, interoperabilidad, seguridad funcional.

Rendimiento
(Eficiencia)

Uso eficiente de recursos: velocidad, capacidad bajo carga, consumo de memoria

Usabilidad

Fácil de aprender, usar y recuperarse de errores. Diseño intuitivo.

Confiabilidad

Funciona sin fallos durante tiempo prolongado: disponibilidad, madurez, recuperación.

Seguridad
(Protección)

Resiste accesos no autorizados: autenticación, cifrado, integridad de datos.

Mantenibilidad

Fácil de modificar, corregir o mejorar: modularidad, reusabilidad, análisis de errores.

Portabilidad

Se adapta a diferentes entornos: instalabilidad, compatibilidad, reemplazo.



1.1 Conceptos de Calidad de Software



¿Cómo Medimos la Calidad del Software?

Pruebas Automatizadas

- Cobertura de código (% de líneas probadas)
- Número de bugs encontrados vs. resueltos

Métricas Técnicas

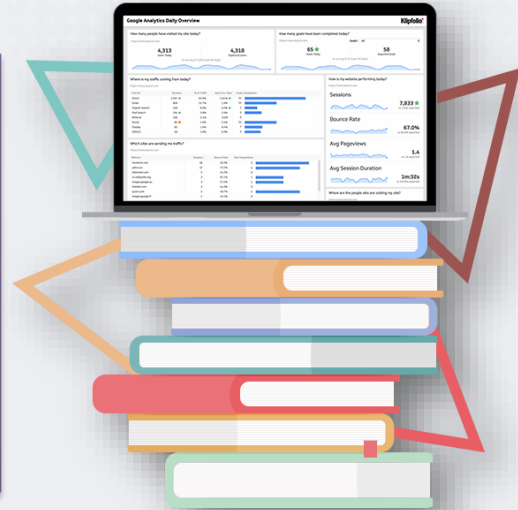
- Tiempo de respuesta (< 2 segundos ideal)
- Uso de CPU/memoria bajo condiciones máximas
- Rendimiento en pruebas de carga (ej. 1000 usuarios simultáneos)

Prácticas de Desarrollo

- Integración continua (CI/CD)
- Revisiones de código (code reviews)
- Uso de estándares (SOLID, limpieza de código)

Experiencia del Usuario (UX)

- Tiempo promedio para completar una tarea
- Tasa de abandono en procesos clave
- Encuestas de satisfacción (NPS, CSAT)



Gracias

Preguntas?



dlgamboal@espe.edu.ec ESPE

